

Guía de Aprendizaje: Línea de Comandos Básica en Linux

Malak Sanchez

Workshop 2026-II

21 de agosto de 2026

1. Motivación

La línea de comandos (*Command Line Interface* o CLI) es la herramienta fundamental para interactuar con sistemas operativos basados en Unix y Linux. A diferencia de las interfaces gráficas (GUI), la línea de comandos permite ejecutar tareas de forma más rápida, precisa y automatizable. Para el desarrollo en C y la programación de sistemas operativos, el dominio de la interfaz de línea de comandos resulta imprescindible para compilar, depurar y gestionar procesos y archivos.

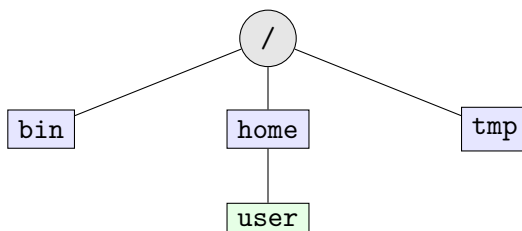
2. Idea Fundamental

Cuando un usuario interactúa con la terminal, no habla directamente con el hardware del computador. Existe una arquitectura en capas: el usuario interactúa con la **Terminal** (emulador de pantalla), la cual transmite texto al **Shell** (intérprete de comandos como Bash). El Shell interpreta el comando introducido, efectúa la llamada al sistema correspondiente y le pide al **Núcleo (Kernel)** que ejecute la acción sobre el hardware.



3. Estructura del Sistema de Archivos y Rutas

En Linux todo se representa mediante un árbol jerárquico unificado que comienza en la raíz /. No existen letras de unidad como en otros sistemas (ej. C:).



- **Ruta Absoluta:** Ubicación completa desde la raíz /, e.g. /home/user/app.
- **Ruta Relativa:** Especifica la ubicación respecto al directorio actual. Usos clave:

- `.` se refiere al directorio actual.
- `..` se refiere al directorio padre.
- `~` se refiere al directorio personal del usuario (*home*).

4. Sintaxis General de un Comando

La sintaxis general de cualquier comando en Linux sigue la estructura:

```
comando [opciones] [argumentos]
```

- **Comando:** Nombre de la utilidad a ejecutar (ej. `ls`).
- **Opciones:** Modifican el comportamiento del comando. Suelen iniciar con `-` (forma corta) o `--` (forma larga). Ejemplos: `-l`, `-all`.
- **Argumentos:** El objeto sobre el cual opera el comando (ej. rutas de archivos o carpetas).

5. Comandos Esenciales

5.1. Navegación e Inspección de Directorios

- `pwd`: Imprime la ruta del directorio de trabajo actual (*Print Working Directory*).
- `ls`: Lista el contenido del directorio.
 - `ls -l`: Formato detallado (permisos, propietario, tamaño, fecha).
 - `ls -a`: Muestra todos los archivos, incluyendo archivos ocultos (los que inician con `.`).
 - `ls -la`: Combina ambas opciones.
- `cd <dir>`: Cambia el directorio actual (*Change Directory*).

5.2. Manipulación de Archivos y Carpetas

- `mkdir <nombre>`: Crea un nuevo directorio.
- `touch <archivo>`: Crea un archivo vacío o actualiza la fecha de modificación.
- `cp <origen> <destino>`: Copia un archivo. Para copiar directorios recursivamente se usa `cp -r`.
- `mv <origen> <destino>`: Mueve o renombra un archivo o directorio.
- `rm <archivo>`: Elimina un archivo. Para eliminar un directorio y su contenido se utiliza `rm -r`.

5.3. Visualización y Búsqueda de Contenido

- `cat <archivo>`: Muestra todo el contenido de un archivo en pantalla.
- `head -n <k> <archivo>`: Muestra las primeras k líneas de un archivo.
- `tail -n <k> <archivo>`: Muestra las últimas k líneas de un archivo.
- `grep «patron»«archivo»`: Busca líneas que coincidan con un patrón de texto.

6. Redirecciones y Tuberías (*Pipes*)

Todo proceso en Linux maneja tres flujos de datos estándar:

1. **Entrada Estándar (`stdin`)**: Descriptor 0 (teclado por defecto).
2. **Salida Estándar (`stdout`)**: Descriptor 1 (pantalla por defecto).
3. **Error Estándar (`stderr`)**: Descriptor 2 (pantalla por defecto).

6.1. Operadores de Redirección

- `comando > archivo`: Redirige la salida estándar a un archivo (sobreescribe).
- `comando » archivo`: Redirige la salida estándar a un archivo (añade al final).
- `comando < archivo`: Toma la entrada estándar desde un archivo.

6.2. Tuberías (*Pipes*)

El operador de tubería `|` conecta la salida estándar de un comando directamente a la entrada estándar de otro comando:

```
comando1 | comando2
```

7. Ejemplo Mínimo Funcional

A continuación se ilustra un flujo completo donde se crea una carpeta de trabajo, un archivo fuente en C, se compila y se ejecuta redirigiendo su salida:

```
1 # 1. Crear carpeta de trabajo y acceder a ella
2 mkdir lab1
3 cd lab1
4
5 # 2. Crear un archivo fuente simple en C
6 cat << 'EOF' > main.c
7 #include <stdio.h>
8
9 int main() {
10     printf("Hello from Linux CLI\n");
11     return 0;
12 }
13 EOF
14
15 # 3. Compilar el programa con gcc
16 gcc -Wall main.c -o program
17
18 # 4. Ejecutar el programa y redirigir la salida a un archivo
19 ./program > output.txt
20
21 # 5. Verificar el contenido generado
22 cat output.txt
```

8. Explicación Paso a Paso

1. `mkdir lab1`: Crea el directorio `lab1` en la ubicación actual.
2. `cd lab1`: Cambia el directorio de trabajo a la carpeta recién creada.
3. `cat « 'EOF' > main.c`: Utiliza un bloque *Here Document* para escribir código C directamente en `main.c`.
4. `gcc -Wall main.c -o program`: Invoca el compilador GCC. La opción `-Wall` activa todos los avisos de advertencia y `-o program` define el nombre del ejecutable resultante.
5. `./program > output.txt`: Ejecuta el archivo ejecutable `program` ubicado en el directorio actual (`./`) y envía el resultado al archivo `output.txt`.
6. `cat output.txt`: Imprime en la consola el texto guardado en `output.txt`.

9. Patrones Comunes de Uso

9.1. Filtrado e Inspección Combinada

Es muy común combinar comandos utilizando tuberías para analizar información:

```
1 # Listar archivos ordenados y buscar aquellos con extension .c
2 ls -la | grep "\.c"
3
4 # Ver los ultimos procesos ejecutados filtrando por nombre
5 ps aux | grep "gcc"
```

9.2. Gestión de Permisos Basica

En Linux, los permisos se dividen en Lectura (**r**), Escritura (**w**) y Ejecución (**x**):

```
1 # Otorgar permiso de ejecucion a un script
2 chmod +x script.sh
```

10. Errores Comunes

- **No such file or directory:** Se intenta acceder a un archivo o carpeta cuyo nombre está mal escrito o cuya ruta es incorrecta.
- **Permission denied:** El usuario no posee los permisos de lectura, escritura o ejecución requeridos para la operación.
- **command not found:** El comando tipeado no está instalado o no se encuentra registrado en la variable de entorno `$PATH`.
- **Olvidar el prefijo ./:** Intentar ejecutar un programa en el directorio actual con `program` en lugar de `./program`.

11. Buenas Prácticas

- **Usar Autocompletado:** Presione la tecla `Tab` para autocompletar nombres de comandos, archivos y rutas.
- **Consultar Manuales:** Utilice `man <comando>` o `<comando> -help` para leer la documentación de cualquier herramienta.
- **Navegar el Historial:** Utilice las teclas de flecha arriba/abajo o la combinación `Ctrl + R` para buscar comandos ejecutados previamente.
- **Tener Cuidado con `rm -rf`:** Este comando elimina archivos y directorios de forma irreversible sin pedir confirmación.

12. Ejercicios

1. Escriba los comandos necesarios para crear una estructura de carpetas llamada `proyecto/src` y `proyecto/bin`.
2. Cree un archivo `datos.txt` que contenga 10 líneas de texto. Utilice los comandos `head` y `tail` junto con una tubería para mostrar únicamente las líneas 4 a la 7.

3. Escriba un programa mínimo en C que imprima un mensaje, compílelo generando el ejecutable `app` y ejecútelo verificando que la salida se guarde correctamente en `resultado.log`.